# Practical 3

## Object Class Methods: toString, equals, hashCode, clone & Nested Classes

### Part A — Practice Programs (Hour 1)

**1) Distinct Points — toString, equals, hashCode**

Code

**Point.java**

```
import java.util.Objects;

public class Point {
    private int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return x == p.x && y == p.y;
    }

    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

**Driver.java**

```
public class Driver {
    public static void main(String[] args) {
        Point[] pts = { new Point(1, 2), new Point(3, 4), new Point(1, 2), new Point(5, 6), new
            Point(3, 4) };
        int distinct = 0;
        for (int i = 0; i < pts.length; i++) {
            boolean seen = false;
            for (int j = 0; j < i; j++) {
                if (pts[i].equals(pts[j])) { seen = true; break; }
            }
            System.out.println(pts[i] + (seen ? " -> repeat" : " -> new"));
            if (!seen) distinct++;
        }
        System.out.println("Distinct: " + distinct);
    }
}
```

Output

```
(1, 2) -> new
(3, 4) -> new
(1, 2) -> repeat
(5, 6) -> new
(3, 4) -> repeat
Distinct: 3
```

**2) Duplicate Card Check — equals/hashCode by rank + suit**

Code

**Card.java**

```
import java.util.Objects;

public class Card {
    private String rank, suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String toString() {
        return rank + " of " + suit;
    }

    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Card)) return false;
        Card c = (Card) o;
        return rank.equals(c.rank) && suit.equals(c.suit);
    }

    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

**Driver.java**

```
public class Driver {
    public static void main(String[] args) {
        Card[] incoming = {
            new Card("Ace", "Spades"), new Card("Queen", "Hearts"),
            new Card("Ten", "Clubs"), new Card("Ace", "Spades"), new Card("Two", "Diamonds")
        };
        Card[] deck = new Card[incoming.length];
        int count = 0;
        for (Card c : incoming) {
            boolean dup = false;
            for (int i = 0; i < count; i++) {
                if (deck[i].equals(c)) { dup = true; break; }
            }
            if (dup) {
                System.out.println("Duplicate found: " + c);
                break;
            }
            deck[count++] = c;
            System.out.println("Added: " + c);
        }
    }
}
```

Output

```
Added: Ace of Spades
Added: Queen of Hearts
Added: Ten of Clubs
Duplicate found: Ace of Spades
```

**3) Fractions (For Advanced Learners — home) — reduce to lowest terms with gcd**

Code

**Fraction.java**

```
import java.util.Objects;

public class Fraction {
    private int num, den;

    public Fraction(int num, int den) {
        int g = gcd(Math.abs(num), Math.abs(den));
        this.num = num / g;
        this.den = den / g;
    }

    private static int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    public String toString() {
        return num + "/" + den;
    }

    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Fraction)) return false;
        Fraction f = (Fraction) o;
        return num == f.num && den == f.den;
    }

    public int hashCode() {
        return Objects.hash(num, den);
    }
}
```

**Driver.java**

```
public class Driver {
    public static void main(String[] args) {
        Fraction a = new Fraction(1, 2);
        Fraction b = new Fraction(2, 4);
        Fraction c = new Fraction(3, 6);
        System.out.println("1/2 -> " + a + ", 2/4 -> " + b + ", 3/6 -> " + c);
        System.out.println("a.equals(b): " + a.equals(b));
        System.out.println("b.equals(c): " + b.equals(c));
        System.out.println("hashCodes equal: " + (a.hashCode() == c.hashCode()));
        System.out.println("1/2 equals 1/3: " + a.equals(new Fraction(1, 3)));
    }
}
```

Output

```
1/2 -> 1/2, 2/4 -> 1/2, 3/6 -> 1/2
a.equals(b): true
b.equals(c): true
hashCodes equal: true
1/2 equals 1/3: false
```

## Part B — Project Work (Hour 2): MiniBank

Account objects now print as readable text and are compared by account number. Customer gets a static nested Address class and a clone() method. Practical 12 depends on correct equals/hashCode when accounts go into collections.

**1-2) Account — toString(), equals(Object) and hashCode() based on accountNumber**

Code

```
import java.util.Objects;

public class Account {
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;
    private static int counter = 0;

    public Account(String ownerName, long openingBalance) {
        counter++;
        this.accountNumber = String.format("AC%04d", counter);
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    // copy constructor used only to demonstrate equals() on same account number
    private Account(Account other) {
        this.accountNumber = other.accountNumber;
        this.ownerName = other.ownerName;
        this.balance = other.balance;
        this.active = other.active;
    }

    public Account copy() { return new Account(this); }

    public void deposit(long amount) { if (amount > 0) balance += amount; }

    public boolean withdraw(long amount) {
        if (amount <= 0 || amount > balance) return false;
        balance -= amount;
        return true;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    public long getBalance() { return balance; }
    public boolean isActive() { return active; }

    public String toString() {
        return accountNumber + " | " + ownerName + " | " + balance + " | " + (active ? "active" :
            "closed");
    }

    // supplementary: multi-line statement
    public String statement() {
        return "---- Statement ----\n"
             + "Account : " + accountNumber + "\n"
             + "Owner   : " + ownerName + "\n"
             + "Balance : " + balance + "\n"
             + "Status  : " + (active ? "active" : "closed") + "\n"
             + "--------------------";
    }

    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Account)) return false;
        Account other = (Account) o;
        return accountNumber.equals(other.accountNumber);
    }

    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

*Supplementary problems covered: toString() shows whether the account is active, and statement() returns a multi-line statement string. @Override is added later, in Practical 7.*

**3-4) Customer — static nested Address class and clone() (implements Cloneable)**

Code

```
public class Customer implements Cloneable {
    public static class Address {
        private String line, city, pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine() { return line; }
        public String getCity() { return city; }
        public String getPincode() { return pincode; }

        public String toString() { return line + ", " + city + " - " + pincode; }
    }

    private final String customerId;
    private String name, email, mobile;
    private Address address;
    private static long customerCounter = 100;

    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }

    public Customer(String name, String email, String mobile, Address address) {
        this.customerId = generateCustomerId();
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.address = address;
    }

    private Customer(Customer o) {
        this.customerId = o.customerId;
        this.name = o.name;
        this.email = o.email;
        this.mobile = o.mobile;
        this.address = new Address(o.address.getLine(), o.address.getCity(),
            o.address.getPincode());
    }

    public Customer clone() {
        return new Customer(this);
    }

    public String getCustomerId() { return customerId; }
    public String getName() { return name; }
    public Address getAddress() { return address; }

    public String toString() {
        return customerId + " | " + name + " | " + address;
    }
}
```

**5) main — toString, equals, instanceof, nested class and clone**

Code

```
public class MiniBank {
    public static void main(String[] args) {
        Account a1 = new Account("Riya", 5000);
        a1.deposit(2000);
        a1.withdraw(3000);
        Account a2 = a1.copy();              // same accountNumber as a1
        Account a3 = new Account("Aman", 2000);

        System.out.println("--- toString() ---");
        System.out.println(a1);
        System.out.println(a3);

        System.out.println("--- equals() / hashCode() ---");
        System.out.println("a1.equals(a2): " + a1.equals(a2));
        System.out.println("a1.equals(a3): " + a1.equals(a3));
        System.out.println("hashCodes of a1 and a2 equal: " + (a1.hashCode() == a2.hashCode()));

        System.out.println("--- instanceof ---");
        Object o = a1;
        System.out.println("o instanceof Account: " + (o instanceof Account));
        Object s = "text";
        System.out.println("s instanceof Account: " + (s instanceof Account));

        System.out.println("--- Nested Address + clone() ---");
        Customer c1 = new Customer("Riya", "riya@example.com", "9876543210",
                new Customer.Address("12 MG Road", "Surat", "395001"));
        Customer c2 = c1.clone();
        System.out.println("Original: " + c1);
        System.out.println("Clone   : " + c2);
        System.out.println("Same object? " + (c1 == c2) + ", same id? " +
            c1.getCustomerId().equals(c2.getCustomerId()));

        System.out.println("--- Statement ---");
        System.out.println(a1.statement());
    }
}
```

Output

```
--- toString() ---
AC0001 | Riya | 4000 | active
AC0002 | Aman | 2000 | active
--- equals() / hashCode() ---
a1.equals(a2): true
a1.equals(a3): false
hashCodes of a1 and a2 equal: true
--- instanceof ---
o instanceof Account: true
s instanceof Account: false
--- Nested Address + clone() ---
Original: CUST101 | Riya | 12 MG Road, Surat - 395001
Clone   : CUST101 | Riya | 12 MG Road, Surat - 395001
Same object? false, same id? true
--- Statement ---
---- Statement ----
Account : AC0001
Owner   : Riya
Balance : 4000
Status  : active
-------------------
```

## Package layout, build and tag

Code

```
minibank/
 |-- Account.java
 |-- Customer.java
 +-- MiniBank.java

javac *.java
java MiniBank
cd ..
git add .
git commit -m "Practical 3 - Object class methods and nested classes"
git tag lab-03
```

*Two accounts with the same accountNumber are reported equal and have the same hashCode, toString() output is readable, and instanceof returns true only for an Account.*

## Key Questions / Analysis / Interpretation

### 1. Why must equals() and hashCode() be overridden together?

The contract says: if two objects are equal according to equals(), they must return the same hashCode(). Hash-based collections (HashMap, HashSet) first use hashCode() to pick a bucket and then equals() to find the match. If only equals() is overridden, two equal accounts can land in different buckets, so a HashSet would hold duplicates and map look-ups would fail.

**2. What does the default Object.toString() return, and why override it?**

It returns the class name, an @ sign and the hash code in hexadecimal (for example Account@1b6d3586), which tells nothing about the object. Overriding it gives a readable description such as AC0001 | Riya | 4000, useful for printing, logging and debugging.

**3. What is the difference between a static nested class and an inner class?**

A static nested class (like Customer.Address) does not hold a reference to an outer object, so it can be created without a Customer: new Customer.Address(...). An inner (non-static) class is tied to an instance of the outer class and can use that instance's fields, so it can only be created through an outer object.